

第6章 计算机的运算方法

- 无论对于那种应用环境，信息在计算机内部的存在形式都是0和1组成的各种二进制编码
- 本章主要介绍参与运算的各类数据（无符号数/有符号数、定点数/浮点数）在机器内部的存在形式，以及在计算机中的算术运算方法
- 使读者理解在计算机自动解题过程中，数据信息的加工处理流程，从而加深对ALU硬件组成及计算机整机工作原理的理解



本章内容与结构

6.1 无符号数和有符号数

6.2 数的定点表示和浮点表示

6.3 定点运算

6.4 浮点四则运算

6.5 算术逻辑单元



6.1 无符号数和有符号数

一、无符号数 没有正负符号，所有二进制位用来存储数值

计算机中参与运算的数都放在寄存器，寄存器的位数即机器字长

机器字长反映无符号数的表示范围



8 位

0 ~ 255



16 位

0 ~ 65535



二、有符号数 符号的正负状态用0/1区分，占用一位

1. 机器数与真值

真值

带符号的数 / 手写

+ 0.1011 (小数)

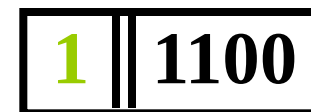
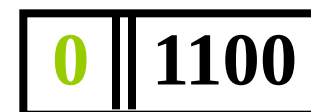
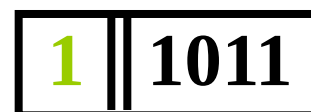
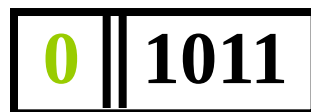
- 0.1011

+ 1100 (整数)

- 1100

机器数

符号数字化的数 / 存储



符号放在有效数字前面

小数点的位置
小数点位置隐含

小数点的位置

小数点的位置

小数点的位置

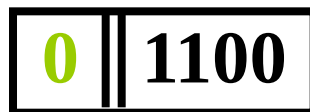


- 符号数字化的数称为机器数
- 按照一定的规则，机器数的符号位与数值部分可以形成各种**编码**，如原码、补码、反码、移码等
- 运算过程中，**符号位能否和数值部分一起参加运算，参加运算时符号位要做哪些处理**，这些与特定的编码有关

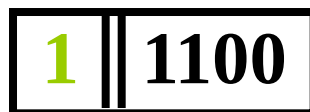
2. 原码表示法

- 原码是机器数中最简单的一种表现形式，也称带符号的绝对值表示；
- 符号位：0表示正数，1表示负数；
- 数值位：是真值的绝对值；

+ 1100



- 1100



原码表示法

(1) 定义

整数

$$[x]_{\text{原}} = \begin{cases} 0, & x > 0 \\ 2^n - x, & x < 0 \end{cases}$$

x 为真值

n 为整数的位数

如

$x = +1110$

$[x]_{\text{原}} = 0, 1110$

用 逗号 将符号位和数值部分隔开

$x = -1110$

$[x]_{\text{原}} = 2^4 + 1110 = 1, 1110$

带符号的绝对值表示



小数

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases}$$

x 为真值

如 $x = +0.1101$ $[x]_{\text{原}} = 0.1101$ 用 **小数点** 将符号位和数值部分隔开

$x = -0.1101$ $[x]_{\text{原}} = 1 - (-0.1101) = 1.1101$

$x = +0.1000000$ $[x]_{\text{原}} = 0.1000000$ 用 **小数点** 将符号位和数值部分隔开

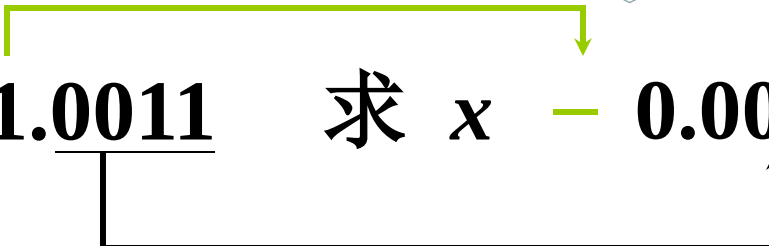
$x = -0.1000000$ $[x]_{\text{原}} = 1 - (-0.1000000) = 1.1000000$



(2) 举例

用概念求解比
用定义简单

例 6.1 已知 $[x]_{\text{原}} = 1.0011$ 求 $x - 0.0011$



解：由定义得

$$x = 1 - [x]_{\text{原}} = 1 - 1.0011 = -0.0011$$

例 6.2 已知 $[x]_{\text{原}} = 1,1100$ 求 $x - 1100$



解：由定义得

$$x = 2^4 - [x]_{\text{原}} = 10000 - 1,1100 = -1100$$



例 6.3 已知 $[x]_{\text{原}} = 0.1101$ 求 x

解：根据定义 $\because [x]_{\text{原}} = 0.1101$

$$\therefore x = +0.1101$$

例 6.4 求 $x = 0$ 的原码

解：设 $x = +0.0000$ $[+0.0000]_{\text{原}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{原}} = 1.0000$

同理，对于整数 $[+0]_{\text{原}} = 0,0000$

$[-0]_{\text{原}} = 1,0000$

$\therefore [+0]_{\text{原}} \neq [-0]_{\text{原}}$



原码的特点：简单、直观

但是用原码作加法时，会出现如下问题：

要求	数1	数2	实际操作	结果符号
加法	正	正	加	正
加法	正	负	减	可正可负
加法	负	正	减	可正可负
加法	负	负	加	负

能否 只作加法？

找到一个与负数等价的正数 来代替这个负数

就可使 减 $\longrightarrow$ 加



6. 补码表示法

(1) 补数的概念

时钟：从6点
调到3点

逆时针

$$\begin{array}{r} 6 \\ - 3 \\ \hline 3 \end{array}$$

顺时针

$$\begin{array}{r} 6 \\ + 9 \\ \hline 15 \\ - 12 \\ \hline 3 \end{array}$$

可见 -3 可用 $+9$ 代替 减法 $\rightarrow$ 加法
称 $+9$ 是 -3 以 12 为模的 补数

记作 $-3 \equiv +9 \pmod{12}$

同理 $-4 \equiv +8 \pmod{12}$

$-5 \equiv +7 \pmod{12}$

时钟以
12为模



- 只要确定了模，就可以找到一个与负数等价的正数来代替这个负数，这个正数就是此负数的补数，这样就可以把减法运算用加法实现；
- 例：设 $A=9$ ， $B=5$ ，求 $A-B \pmod{12}$
对于模12来说，-5可以用其补数+7代替
所以 $A-B=9+7=16$ 而16等价于4

结论

- 一个负数加上“模”即得该负数的补数
- 一个正数和一个负数互为补数时，它们绝对值之和即为模数，二进制数也符合上述规律

• 数字逻辑电路中的四位可逆计数器（模 16） $1011 \longrightarrow 0000$?

$$\begin{array}{r} 1011 \\ - 1011 \\ \hline 0000 \end{array}$$

$$\begin{array}{r} 1011 \\ + 0101 \\ \hline 10000 \end{array}$$

自然去掉

可见 -1011 可用 $+0101$ 代替

记作 $-1011 \equiv +0101 \pmod{2^4}$

同理 $-011 \equiv +101 \pmod{2^3}$

$-0.1001 \equiv +1.0111 \pmod{2}$



(2) 正数的补数即为其本身

两个互为补数的数
分别加上模
结果仍互为补数

$$\begin{array}{rcl} -1011 & \equiv & +0101 \pmod{2^4} \\ +10000 & & +10000 \\ \hline +0101 & \equiv & +10101 \end{array}$$

$\therefore +0101 \equiv +0101 \pmod{2^4}$ 丢掉

- 把补数的概念用于计算机的机器数，就出现了补码
- 计算机中的存储单元、寄存器有固定的n位二进制代码，也存在着类似于模2的n次方的特性



(3) 补码定义

整数

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

x 为真值

n 为整数的位数

如

$$x = +1010$$

$$x = -1011000$$

$$[x]_{\text{补}} = 0,1010$$

用 逗号 将符号位
和数值部分隔开

$$\begin{array}{r} [x]_{\text{补}} = 2^{7+1} + (-1011000) \\ = 100000000 \\ - \quad 1011000 \\ \hline 1,0101000 \end{array}$$



小数

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

x 为真值

如

$$x = +0.1110$$

$$x = -0.1100000$$

$$[x]_{\text{补}} = 0.1110$$

$$[x]_{\text{补}} = 2 + (-0.1100000)$$

$$= 10.0000000$$

$$- 0.1100000$$

$$\hline 1.0100000$$

用 小数点 将符号位
和数值部分隔开



(4) 求补码的快捷方式

设 $x = -1010$ 时

$$\begin{aligned} \text{则 } [x]_{\text{补}} &= 2^{4+1} - 1010 &= 11111 + 1 - 1010 \\ &= 100000 &= 11111 + 1 \\ &\quad - 1010 &\quad - 1010 \\ \hline &= 1,0110 &\hline &\quad \boxed{10101} + 1 \\ & &= 1,0110 \end{aligned}$$

$$\text{又 } [x]_{\text{原}} = \boxed{1,1010}$$

当真值为 负 时，补码 可用 原码除符号位外
每位取反，末位加 1 求得



(5) 举例

例 6.5 已知 $[x]_{\text{补}} = 0.0001$

求 x

解：由定义得 $x = +0.0001$

若能直接由补码
给出原码，真值
更容易得到

例 6.6 已知 $[x]_{\text{补}} = 1.0001$ $[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}}$

求 x

$$[x]_{\text{原}} = 1.1111$$

解：由定义得

$$\therefore x = -0.1111$$

$$\begin{aligned} x &= [x]_{\text{补}} - 2 \\ &= 1.0001 - 10.0000 \\ &= -0.1111 \end{aligned}$$

可以看出，负小数的补码，数值位按位求反，末尾加一，就是原码的数值位



例 6.7 已知 $[x]_{\text{补}} = 1,1110$

求 x

解：由定义得

$$\begin{aligned}x &= [x]_{\text{补}} - 2^{4+1} \\&= 1,1110 - 100000 \\&= -0010\end{aligned}$$

$$[x]_{\text{补}} \xrightarrow{?} [x]_{\text{原}}$$

$$[x]_{\text{原}} = 1,0010$$

$$\therefore x = -0010$$

当真值为 负 时，原码 可用 补码除符号位外
每位取反，末位加 1 求得



练习 求下列真值的补码

负数原码数值位，从右往左至第一个1保持不变，高位全部取反，得到补码的数值位，反之亦然

真值	$[x]_{\text{补}}$	$[x]_{\text{原}}$
$x = +70 = 1000110$	0, 1000110	0, 1000110
$x = -70 = -1000110$	1, 0111010	1, 1000110
$x = 0.1110$	0.1110	0.1110
$x = -0.1110$	1.0010	1.1110
$x = \boxed{0.0000} \quad [+0]_{\text{补}} = [-0]_{\text{补}}$	$\boxed{0.0000}$	0.0000
$x = \boxed{-0.0000}$	$\boxed{0.0000}$	1.0000
$x = -1.0000$	1.0000	不能表示

由小数补码定义
$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2 + x & 0 > x \geq -1 \pmod{2} \end{cases}$$

$$[-1]_{\text{补}} = 2 + x = 10.0000 - 1.0000 = 1.0000$$



4. 反码表示法

(1) 定义

整数

$$[x]_{\text{反}} = \begin{cases} 0, & x & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \pmod{2^{n+1} - 1} \end{cases}$$

x 为真值

n 为整数的位数

如

$$x = +1101$$

$$x = -1101$$

$$[x]_{\text{反}} = 0,1101$$

$$[x]_{\text{反}} = (2^{4+1} - 1) - 1101$$

$$= 11111 - 1101$$

$$= 1,0010$$

用逗号将符号位
和数值部分隔开

负整数反码的数值位等于真值的绝对值按位求反



小数

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2 - 2^{-n}) + x & 0 \geq x > -1 \pmod{2 - 2^{-n}} \end{cases}$$

x 为真值 n 为小数的位数

如

$$x = + 0.1101$$

$$x = - 0.1010$$

$$[x]_{\text{反}} = 0.1101$$

$$[x]_{\text{反}} = (2 - 2^{-4}) - 0.1010$$

$$= 1.1111 - 0.1010$$

$$= 1.0101$$

用 小数点 将符号位
和数值部分隔开

负小数反码的数值位等于真值的绝对值按位求反



(2) 举例

例6.8 已知 $[x]_{\text{反}} = 0,1110$ 求 x

解: 由定义得 $x = +1110$

例6.9 已知 $[x]_{\text{反}} = 1,1110$ 求 x

解: 由定义得
$$\begin{aligned} x &= [x]_{\text{反}} - (2^{4+1} - 1) \\ &= 1,1110 - 11111 \\ &= -0001 \end{aligned}$$

负数由反码到真值，数值位按位求反

例 6.10 求 0 的反码

解: 设 $x = +0.0000$ $[+0.0000]_{\text{反}} = 0.0000$

$x = -0.0000$ $[-0.0000]_{\text{反}} = 1.1111$

同理，对于整数 $[+0]_{\text{反}} = 0,0000$ $[-0]_{\text{反}} = 1,1111$

$\therefore [+0]_{\text{反}} \neq [-0]_{\text{反}}$



三种机器数的小结

➤ 最高位为符号位，书写上用 “,”（整数）
或 “.”（小数）将数值部分和符号位隔开

➤ 对于正数，符号位是0，原码 = 补码 = 反码

➤ 对于负数，符号位为1，其数值部分：

原码的数值部分就是真值的绝对值

原码除符号位外每位取反末位加1 → 补码，
反之亦然

原码除符号位外每位取反 → 反码，反之
亦然



例6.11 设**机器数**字长为8位（其中1位为符号位）
 对于整数，当其分别代表无符号数、原码、补码和反码时，对应的**真值范围**各为多少？

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	± 0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0



例6.12 已知 $[y]_{\text{补}}$ 求 $[-y]_{\text{补}}$

解： 设 $[y]_{\text{补}} = y_0 \cdot y_1 y_2 \cdots y_n$

<I> $[y]_{\text{补}} = 0 \cdot y_1 y_2 \cdots y_n$

$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 1 \cdot \bar{y}_1 \bar{y}_2 \cdots \bar{y}_n + 2^{-n}$$

结论也适用于
整数情况

<II> $[y]_{\text{补}} = 1 \cdot y_1 y_2 \cdots y_n$

$[y]_{\text{补}}$ 连同符号位在内， 每位取反， 末位加 1

即得 $[-y]_{\text{补}}$

$$[-y]_{\text{补}} = 0 \cdot \bar{y}_1 \bar{y}_2 \cdots \bar{y}_n + 2^{-n}$$



5. 移码表示法

补码表示很难直观判断其真值大小

如	十进制	二进制	补码	
	$x = +21$	$+10101$	$0,10101$	 错 大
	$x = -21$	-10101	$1,01011$	
	$x = +31$	$+11111$	$0,11111$	 错 大
	$x = -31$	-11111	$1,00001$	

$x + 2^5$

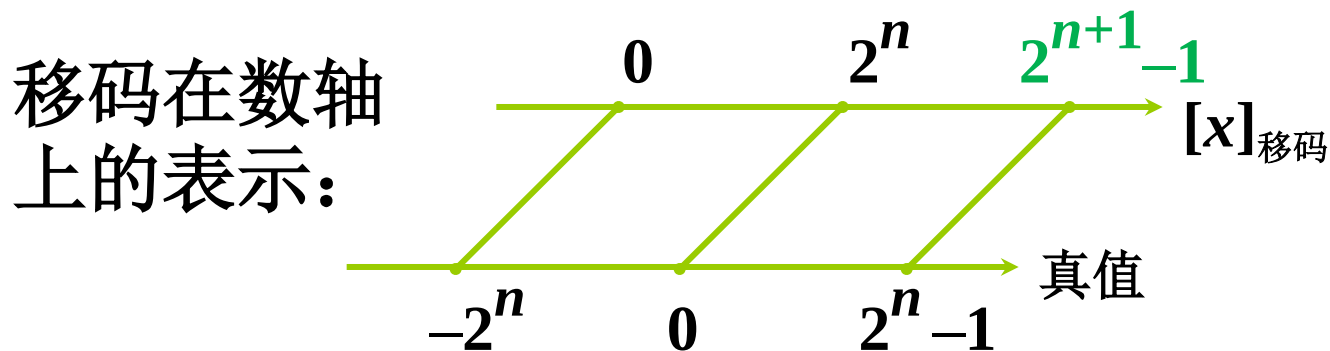
$+10101 + 100000 = 110101$		大 正确
$-10101 + 100000 = 001011$		
$+11111 + 100000 = 111111$		大 正确
$-11111 + 100000 = 000001$		



(1) 移码定义

$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

x 为真值, n 为 整数的位数 (移码只考虑整数, 不分正负)



如 $x = 10100$

$$[x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

$$x = -10100$$

$$[x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

用 逗号 将符号位和数值部分隔开



(2) 移码和补码的比较

设 $x = +1100100$

$$[x]_{\text{移}} = 2^7 + 1100100 = \mathbf{1},1100100$$

$$[x]_{\text{补}} = \mathbf{0},1100100$$

设 $x = -1100100$

$$[x]_{\text{移}} = 2^7 - 1100100 = \mathbf{0},0011100$$

$$[x]_{\text{补}} = \mathbf{1},0011100$$

补码与移码只差一个符号位



(3) 真值、补码和移码的对照表

真值 x ($n=5$)	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的 十进制整数
- 1 0 0 0 0 0	1 0 0 0 0 0	0 0 0 0 0 0	0
- 1 1 1 1 1	1 0 0 0 0 1	0 0 0 0 0 1	1
- 1 1 1 1 0	1 0 0 0 1 0	0 0 0 0 1 0	2
⋮	⋮	⋮	⋮
- 0 0 0 0 1	1 1 1 1 1 1	0 1 1 1 1 1	31
± 0 0 0 0 0	0 0 0 0 0 0	1 0 0 0 0 0	32
+ 0 0 0 0 1	0 0 0 0 0 1	1 0 0 0 0 1	33
+ 0 0 0 1 0	0 0 0 0 1 0	1 0 0 0 1 0	34
⋮	⋮	⋮	⋮
+ 1 1 1 1 0	0 1 1 1 1 0	1 1 1 1 1 0	62
+ 1 1 1 1 1	0 1 1 1 1 1	1 1 1 1 1 1	63



(4) 移码的特点 接上页 $n=5$

➤ 当 $x = 0$ 时 $[+0]_{\text{移}} = 2^5 + 0 = 1,00000$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

$$\therefore [+0]_{\text{移}} = [-0]_{\text{移}}$$

➤ 当 $n = 5$ 时 最小的真值为 $-2^5 = -100000$

$$[-100000]_{\text{移}} = 2^5 - 100000 = 000000$$

可见，最小真值的移码为全 0

用移码表示浮点数的阶码

能方便地判断浮点数的阶码大小



作业题 6.1

- 由真值求原码和补码：

$$X=+1010$$

$$X=-1101$$

$$X=0.1001$$

$$X=-0.0110$$

- 由补码求原码：

$$[x]_{\text{补}} = 1,1110$$

$$[x]_{\text{补}} = 0.0001$$

- 由 $[x]_{\text{补}}$ 求 $[-x]_{\text{补}}$ ：

$$[x]_{\text{补}} = 1,1110$$

$$[x]_{\text{补}} = 0.0001$$

Review: 无符号数和有符号数

- 无符号数
- 有符号数
 - 真值
 - 机器数
 - 原码表示
 - 补码表示
 - 反码表示
 - 移码表示

几种机器数总结

➤ 最高位为符号位，书写上用 “,”（整数）

或 “.”（小数）将数值部分和符号位隔开

➤ 对于正数，原码 = 补码 = 反码

➤ 对于负数，符号位为 1，其数值部分

除符号位外每位取反末位加1：原码 $\longleftrightarrow$ 补码

除符号位外每位取反：原码 $\longleftrightarrow$ 反码

➤ 由 $[y]_{\text{补}}$ 求 $[-y]_{\text{补}}$ ： $[y]_{\text{补}}$ 连同符号位在内，每位取反，末位加 1

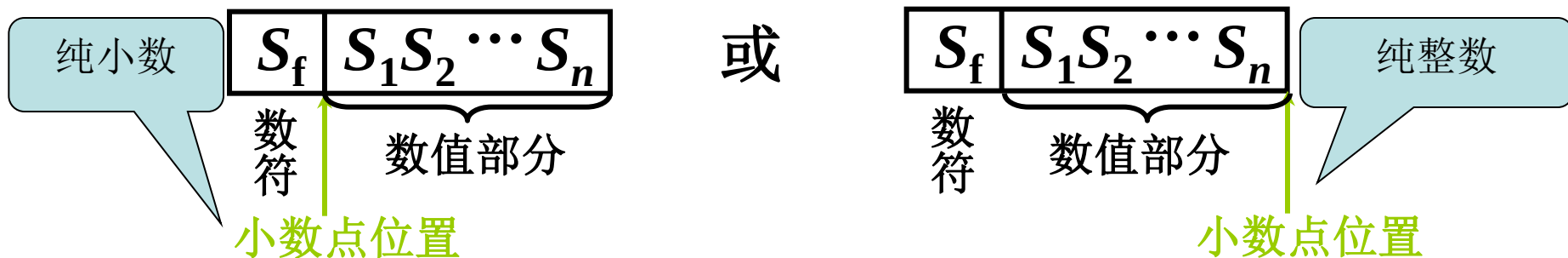
➤ 补码与移码只差一个符号位



6.2 数的定点表示和浮点表示

小数点按约定方式标出，两种方法表示其存在

一、定点表示：小数点固定在某一位置



定点机

小数定点机

整数定点机

原码

$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$

$-(2^n - 1) \sim +(2^n - 1)$

补码

$-1 \sim +(1 - 2^{-n})$

$-2^n \sim +(2^n - 1)$

反码

$-(1 - 2^{-n}) \sim +(1 - 2^{-n})$

$-(2^n - 1) \sim +(2^n - 1)$



为什么采用浮点数：

- 计算机处理的数不一定是纯小数、纯整数，定点数无法表示，如圆周率；
- 而有些数的数值范围相差很大，在字长有限的计算机中，不能直接用定点数表示，如不同物体的质量，需采用科学计数法；
- 浮点数：小数点的位置可以浮动，乘上一个比例因子得到实际值。

二、浮点表示

$N = S \times r^j$ 浮点数的一般形式

S 尾数 j 阶码 r 基数（基值）

计算机中 r 取 2、4、8、16 等

当 $r = 2$ $N = 11.0101$ 二进制表示

✓ $= 0.110101 \times 2^{10}$ 规格化数

$$= 1.10101 \times 2^1$$

$$= 1101.01 \times 2^{-10}$$

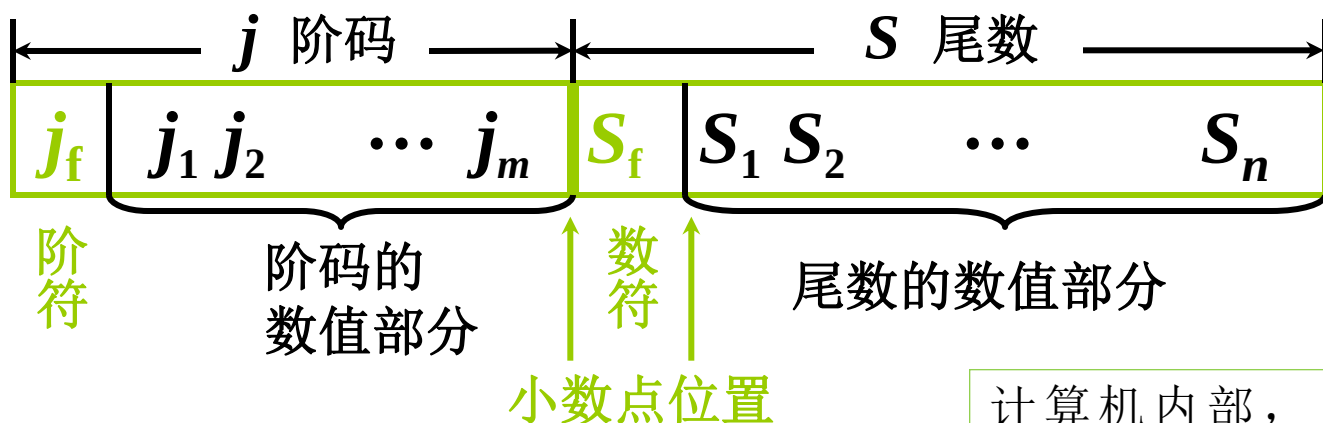
$$✓ = 0.00110101 \times 2^{100}$$

计算机中规定 S 纯小数、可正可负

j 整数、可正可负指示小数点移动方向



1. 浮点数（在机器中）的表示形式



计算机内部，只保存阶码和尾数，基数 r 隐含约定不存储

S_f 代表浮点数的符号

n 其位数反映浮点数的精度

m 其位数反映浮点数的表示范围

j_f 和 m 共同表示小数点移动的方向和次数，定义小数点实际位置



- 为了提高数据精度，计算机中规定浮点数的尾数用纯小数形式
- 基数为2时，尾数最高位是1（小数点后面第一位是1）的浮点数称为规格化浮点数，如： $X = 0.101011 \times 2^{10}$
- 浮点数表示成规格化形式之后，精度最高，因为尾数所有位都有效使用

2. 浮点数的表示范围

假设尾数数值位有 n 位，
阶码数值位有 m 位

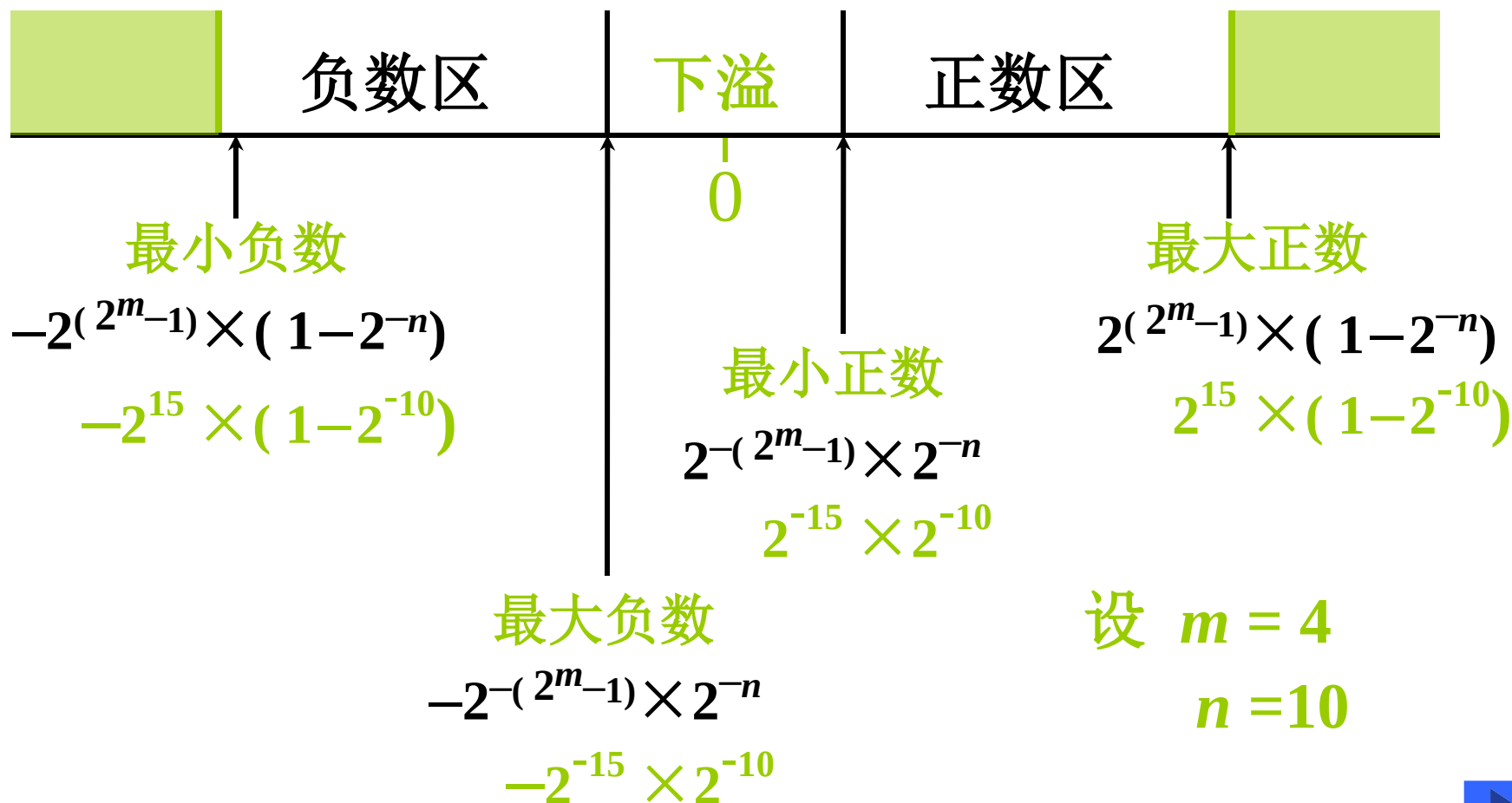
上溢 阶码 $>$ 最大阶码 $(+1111)$

下溢 阶码 $<$ 最小阶码 (-1111)

按 机器零 处理

上溢

上溢



练习

设机器数字长为 24 位，欲表示 ± 3 万的十进制数，试问在保证数的最大精度的前提下，除阶符、数符各取 1 位外，阶码、尾数各取几位？

解： $\because 2^{14} = 16384 \quad 2^{15} = 32768$

$\therefore$ 15 位二进制数可反映 ± 3 万之间的十进制数

$$2^{15} \times 0.\underbrace{\times \times \times \dots \times \times \times}_{? \text{ 位}}$$

$m = 4, 5, 6, \dots$

把机器数的位数尽量留给尾数，以保证最大精度

满足最大精度可取 $m = 4$ ，则 $n = 18$



6. 浮点数的规格化形式

$r = 2$ 尾数最高位为 1

$r = 4$ 尾数最高 2 位不全为 0

$r = 8$ 尾数最高 3 位不全为 0

基数不同，浮点数的
规格化形式不同

4. 浮点数的规格化

$r = 2$ 左规 尾数左移 1 位，阶码减 1

右规 尾数右移 1 位，阶码加 1

$r = 4$ 左规 尾数左移 2 位，阶码减 1

右规 尾数右移 2 位，阶码加 1

$r = 8$ 左规 尾数左移 3 位，阶码减 1

右规 尾数右移 3 位，阶码加 1

基数 r 越大，可表示的浮点数的范围越大

基数 r 越大，浮点数的精度降低



例如：设 $m = 4$, $n = 10$, $r = 2$

尾数规格化后的浮点数表示范围

最大正数 $2^{+1111} \times \underbrace{0.1111111111}_{10 \text{ 个 } 1} = 2^{15} \times (1 - 2^{-10})$

最小正数 $2^{-1111} \times \underbrace{0.1000000000}_{9 \text{ 个 } 0} = 2^{-15} \times 2^{-1} = 2^{-16}$

最大负数
(绝对值最小) $2^{-1111} \times (-\underbrace{0.1000000000}_{9 \text{ 个 } 0}) = -2^{-15} \times 2^{-1} = -2^{-16}$

最小负数
(绝对值最大) $2^{+1111} \times (-\underbrace{0.1111111111}_{10 \text{ 个 } 1}) = -2^{15} \times (1 - 2^{-10})$



三、举例

例 6.13 将 $+\frac{19}{128}$ 写成二进制定点数、浮点数及在定点机和浮点机中的机器数形式。定点数、浮点数尾数均取 10 位数值，数符取 1 位；阶码取 5 位，含 1 位阶符。

解： 设 $x = +\frac{19}{128}$

二进制形式 $x = 0.0010011$

定点表示 $x = 0.0010011\ 000$

浮点规格化形式 $x = 0.1001100000 \times 2^{-10}$

定点机中 $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = 0.0010011000$

浮点机中 $[x]_{\text{原}} = 1, 0010; 0. 1001100000$

浮点数书写形式
阶码（纯整数）；
尾数（纯小数）

$[x]_{\text{补}} = 1, 1110; 0. 1001100000$

$[x]_{\text{反}} = 1, 1101; 0. 1001100000$



例 6.14 将 -58 表示成二进制定点数和浮点数，并写出它在定点机和浮点机中的三种机器数及阶码为移码、尾数为补码的形式（其他要求同上例）。

解： 设 $x = -58$

二进制形式 $x = -111010$

定点表示 $x = -0000111010$

浮点规格化形式 $x = -(0.1110100000) \times 2^{110}$

定点机中

$[x]_{\text{原}} = 1, 0000111010$

$[x]_{\text{补}} = 1, 1111000110$

$[x]_{\text{反}} = 1, 1111000101$

浮点机中

$[x]_{\text{原}} = 0, 0110; 1. 1110100000$

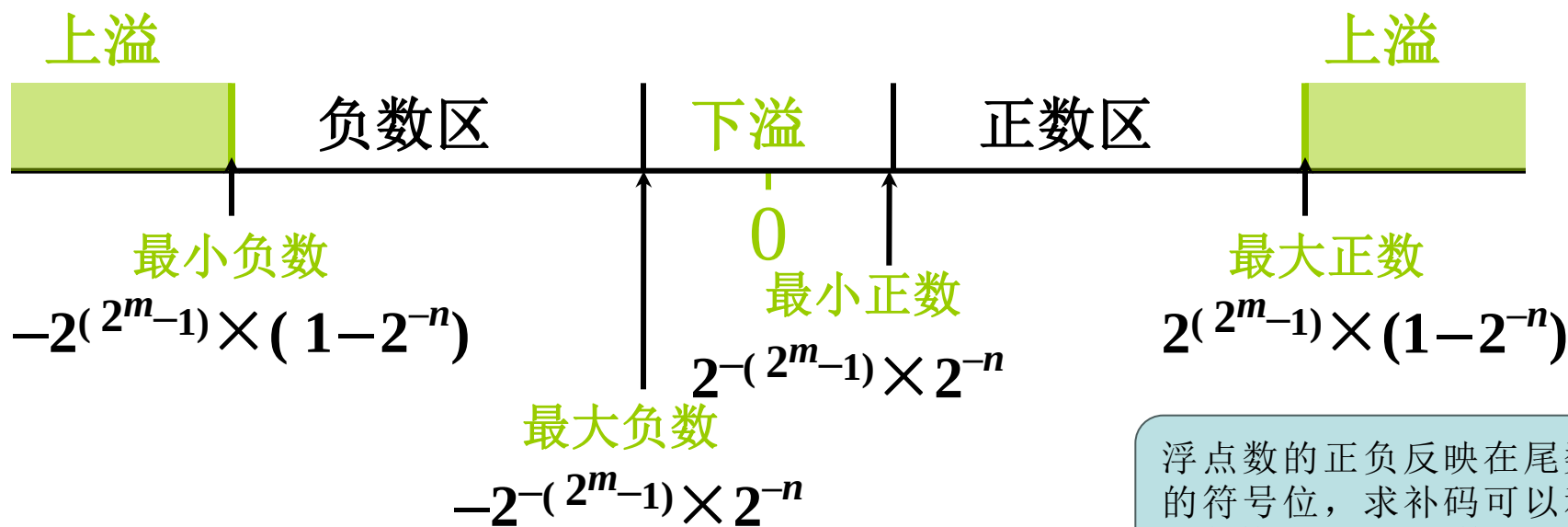
$[x]_{\text{补}} = 0, 0110; 1. 0001100000$

$[x]_{\text{反}} = 0, 0110; 1. 0001011111$

$[x]_{\text{阶移、尾补}} = 1, 0110; 1. 0001100000$



例6.15 写出对应下图所示的浮点数的补码形式。设 $n = 10$, $m = 4$, 阶符、数符各取 1 位。



浮点数的正负反映在尾数的符号位，求补码可以利用负数的补码的规律

解：

真值

补码

最大正数 $2^{15} \times (1-2^{-10})$

0,1111; 0.1111111111

最小正数 $2^{-15} \times 2^{-10}$

1,0001; 0.0000000001

最大负数 $-2^{-15} \times 2^{-10}$

1,0001; 1.1111111111

最小负数 $-2^{15} \times (1-2^{-10})$

0,1111; 1.0000000001



机器零

(1) 当浮点数 **尾数为 0** 时，不论其阶码为何值
按机器零处理

(2) 当浮点数 **阶码等于或小于它所表示的最小
数** 时，不论尾数为何值，按机器零处理

如 $m = 4$ $n = 10$

当阶码和尾数都用补码表示时，机器零为

$\times, \times \times \times \times; \quad \mathbf{0.00 \cdots 0}$

(阶码 = -16) $\mathbf{1, 0000; \quad \times.\times\times \cdots \times}$

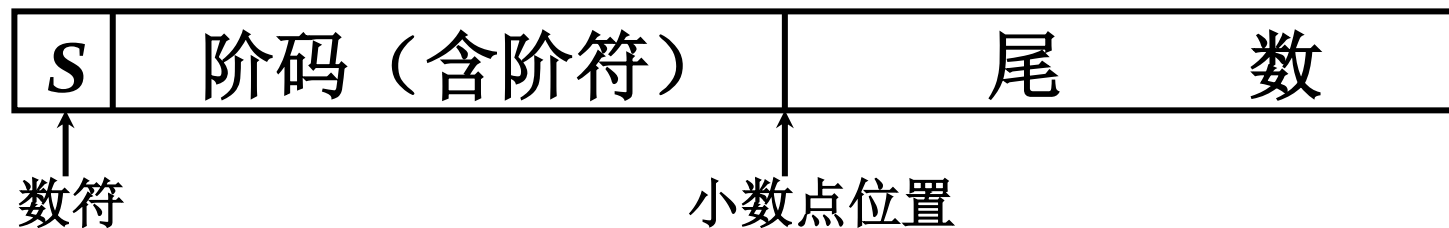
当阶码用移码，尾数用补码表示时，机器零为

$\mathbf{0, 0000; 0.00 \cdots 0}$

有利于机器中 “判 0” 电路的实现



四、IEEE 754 标准



尾数为规格化表示

非“0”的有效位最高位为“1”（隐含）

	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80



Review: 定点数和浮点数

- 定点数：小数点位置固定
 - 纯整数
 - 纯小数
- 浮点数：小数点位置可以浮动
 - 阶码和尾数决定了浮点数的范围及精度
 - 规格化浮点数的概念

作业题 6.2

- 将 -60 、 $\frac{21}{128}$ 表示成二进制定点数和浮点数，并写出它在定点机和浮点机中的三种机器数，定点数、浮点数尾数均取 **10** 位数值，数符取 **1** 位，浮点数阶码取 **5** 位（含**1**位阶符）。

6.3 定点运算

一、移位运算

1. 移位的意义

$$15.\text{m} = 1500.\text{cm}$$

小数点右移 2 位

机器用语 15 相对于小数点 左移 2 位
(小数点不动, 前面补两个 0)

左移 绝对值扩大

右移 绝对值缩小



- 计算机中小数点位置是事先约定的，二进制表示的机器数，在相对于小数点做 n 位左移或者右移时，其实质上就是该数乘以或者除以 2^n
- 在计算机中，移位与加减配合，能够实现乘除运算

- 由于机器字长固定，机器数左移或右移必然会使低位或高位出现空位，对空位应该是补0还是补1呢（对绝对值来说都是补0），这与是否有符号数及编码方式有关
- 有符号数的移位称为算术移位，要求：不改变符号位
- 在移位过程中，数值部分的高位有效位丢失造成错误，低位有效位丢失会影响精度
- 可想而知：无论哪种编码，正数移位规则最为简单，负数中原码表示移位规则简单

2. 算术移位规则

符号位不变

真值	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移 添 0
		右移 添 1
	反 码	1



例6.16

设机器数字长为 8 位（含 1 位符号位），写出 $A = +26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = +26 = +11010$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0,0011010$

移位操作	机 器 数	对应的真值
	$[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}}$	
移位前	0,0011010	+26
左移一位	0,0110100	+52
左移两位	0,1101000	+104
右移一位	0,0001101	+13
右移两位	0,0000110	+6



例6.17

设机器数字长为 8 位（含 1 位符号位），写出 $A = -26$ 时，三种机器数左、右移一位和两位后的表示形式及对应的真值，并分析结果的正确性。

解： $A = -26 = -11010$

原码	移位操作	机 器 数	对应的真值
	移位前	1,0011010	- 26
	左移一位	1,0110100	- 52
	左移两位	1,1101000	- 104
	右移一位	1,0001101	- 13
	右移两位	1,0000110	- 6



补码

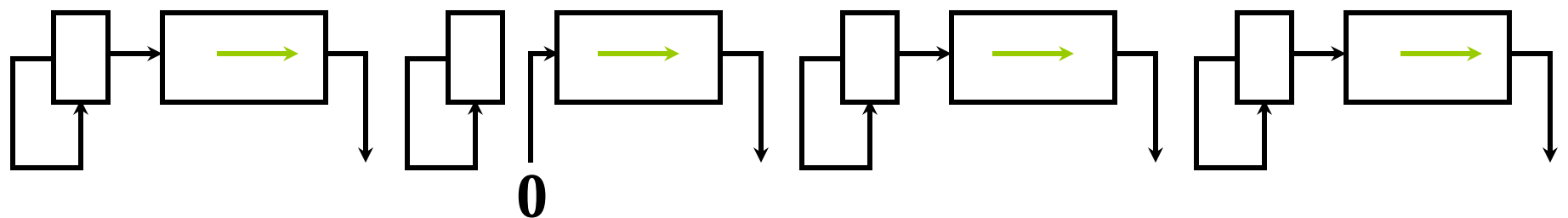
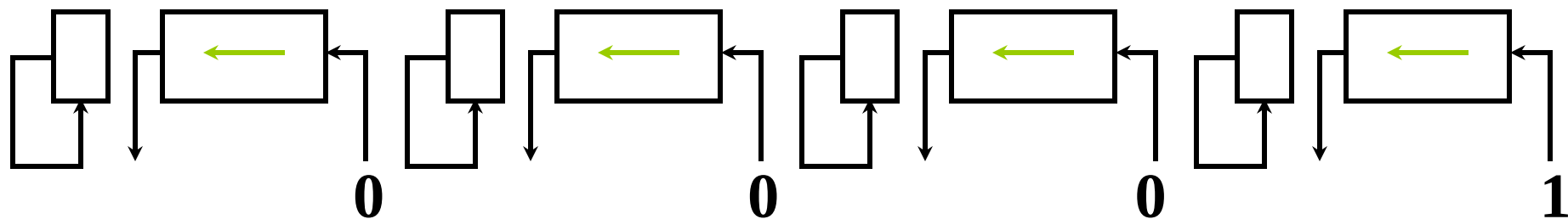
移位操作	机 器 数	对应的真值
移位前	1,1100110	– 26
左移一位	1,1001100	– 52
左移两位	1,0011000	– 104
右移一位	1,1110011	– 13
右移两位	1,1111001	– 7

反码

移位操作	机 器 数	对应的真值
移位前	1,1100101	– 26
左移一位	1,1001011	– 52
左移两位	1,0010111	– 104
右移一位	1,1110010	– 13
右移两位	1,1111001	– 6



3. 算术移位的硬件实现



(a) 真值为正

(b) 负数的原码

(c) 负数的补码

(d) 负数的反码

← 丢 1 出错

出错

正确

正确

→ 丢 1 影响精度

影响精度

影响精度

正确

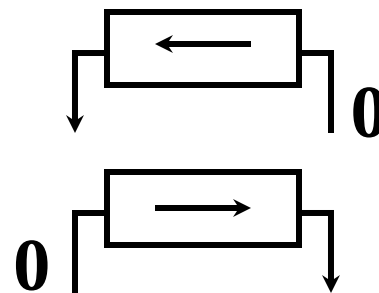
4. 算术移位和逻辑移位的区别

算术移位 有符号数的移位

逻辑移位 无符号数的移位

逻辑左移 低位添 0，高位移丢

逻辑右移 高位添 0，低位移丢



例如 01010011

10110010

逻辑左移 10100110

逻辑右移 01011001

算术左移 00100110

算术右移 11011001 (补码)

为防止高位 1 移丢，可采用带进位的算术移位，符号位移至进位



作业题 6.3

1. 对下列数进行算术左移、右移一位，假设每个数是原码、补码、反码表示：

$$[x]=0.0011010$$

$$[y]=1.0011010$$

2. 对下面的两个数进行逻辑左移、右移一位：

$$[x]=00011010$$

$$[y]=10011010$$

二、加减法运算

- 加减法运算是计算机中最基本的运算
- 减法运算可用加法处理 $A-B=A+(-B)$
- 计算机中用补码做加减法运算

1. 补码加减运算公式

连同符号位一起相加，符号位产生的进位自然丢掉

(1) 加法

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2}$

(2) 减法

$$A-B = A+(-B)$$

整数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2}$



2. 举例

例 6.18 设 $A = 0.1011$, $B = -0.0101$

求 $[A + B]_{\text{补}}$

验证

$$\begin{array}{rcl} \text{解: } [A]_{\text{补}} & = & 0.1011 \\ + [B]_{\text{补}} & = & 1.1011 \\ \hline [A]_{\text{补}} + [B]_{\text{补}} & = & 10.0110 = [A + B]_{\text{补}} \\ \therefore A + B & = & 0.0110 \end{array}$$

$$\begin{array}{r} 0.1011 \\ - 0.0101 \\ \hline 0.0110 \end{array}$$

例 6.19 设 $A = -9$, $B = -5$

求 $[A+B]_{\text{补}}$

验证

$$\begin{array}{rcl} \text{解: } [A]_{\text{补}} & = & 1, 0111 \\ + [B]_{\text{补}} & = & 1, 1011 \\ \hline [A]_{\text{补}} + [B]_{\text{补}} & = & 11, 0010 = [A + B]_{\text{补}} \\ \therefore A + B & = & -1110 \end{array}$$

$$\begin{array}{r} -1001 \\ + -0101 \\ \hline -1110 \end{array}$$



例 6.20 设机器数字长为 8 位（含 1 位符号位）
且 $A = 15$, $B = 24$, 用补码求 $A - B$

解: $A = 15 = 0001111$

$B = 24 = 0011000$

$[A]_{\text{补}} = 0, 0001111$ $[B]_{\text{补}} = 0, 0011000$

$+ [-B]_{\text{补}} = 1, 1101000$

$[A]_{\text{补}} + [-B]_{\text{补}} = 1, 1110111 = [A - B]_{\text{补}}$

$\therefore A - B = -1001 = -9$

练习 1 设 $x = \frac{9}{16}$ $y = \frac{11}{16}$, 用补码求 $x+y$

$x + y = -0.1100 = -\frac{12}{16}$ 错

练习 2 设机器数字长为 8 位（含 1 位符号位）
且 $A = -97$, $B = +41$, 用补码求 $A - B$

$A - B = +1110110 = +118$ 错



- 运算结果超过机器字长所能表示的范围，产生“溢出”
- 加法操作，操作数同号，可产生溢出
- 减法操作，操作数异号，可产生溢出
- 在补码定点加减运算过程中，必须对结果是否溢出做出判断

6. 溢出判断

(1) 一位符号位判溢出

实际做加法的 **两个数**--减法时即为被减数和“求补”以后的减数如 $[-B]_{\text{补}}$ ，如果**补码符号位相同**--实际对应同号加法或异号减法，若其结果的符号与原操作数的符号不同，即为溢出

硬件实现常采用的判别方法：

最高有效位的进位 $\oplus$ 符号位的进位 = 1 溢出

$$\begin{array}{lcl} \text{如} & \begin{array}{l} 1 \oplus 0 = 1 \\ 0 \oplus 1 = 1 \end{array} & \left. \vphantom{\begin{array}{l} 1 \oplus 0 = 1 \\ 0 \oplus 1 = 1 \end{array}} \right\} \text{有 溢出} \\ & \begin{array}{l} 0 \oplus 0 = 0 \\ 1 \oplus 1 = 0 \end{array} & \left. \vphantom{\begin{array}{l} 0 \oplus 0 = 0 \\ 1 \oplus 1 = 0 \end{array}} \right\} \text{无 溢出} \end{array}$$



(2) 两位符号位判溢出

$$[x]_{\text{补}'} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \pmod{4} \end{cases}$$

$$[x]_{\text{补}'} + [y]_{\text{补}'} = [x + y]_{\text{补}'} \pmod{4}$$

$$[x - y]_{\text{补}'} = [x]_{\text{补}'} + [-y]_{\text{补}'} \pmod{4}$$

结果的双符号位 **相同**

未溢出

00, 00. × × × × × × × ×
11, 11. × × × × × × × ×

结果的双符号位 **不同**

溢出

10, 0 × × × × × × × ×
01, 1 × × × × × × × ×

最高符号位 代表其 **真正的符号**



- 采用双符号方案时，任何正确的数，两个符号位的值总是相同的；
- 在寄存器或者主存中的操作数只需要保存一位符号位；
- 相加时一位符号值要同时送到加法器两位符号输入端。

用两位符号位判断

- $X=0.1011$
 $y=0.0011$
- $[x]_{\text{补}}=00.1011$
- $[y]_{\text{补}}=00.0011$
- 00.1011
- $+ 00.0011$
- ---

 00.1110

结果无溢出

- $X=0.1011$ $y=0.1111$
- $[x]_{\text{补}}=00.1011$
- $[y]_{\text{补}}=00.1111$
- 00.1011
- $+ 00.1111$
- ---

 01.1010

结果有溢出

例：设机器数字长为8位（含1位符号位），用补码运算规则计算：

（1） $A=-87$ ， $B=53$ ，求 $A-B$ 。

（2） $A=115$ ， $B=-24$ ，求 $A+B$ 。

解：求A-B

$$A = -87 = -101\ 0111,$$

$$B = 53 = 110\ 101$$

$$[A]_{\text{补}} = 11,010\ 1001,$$

$$[B]_{\text{补}} = 00,011\ 0101,$$

$$[-B]_{\text{补}} = 11,100\ 1011$$

$$[A-B]_{\text{补}} = 11,0101001$$

$$+ 11,1001011$$

$$= 110,1110100 \text{ ---溢出}$$

求A+B

A=115= 1110011,

B= -24= -0011000

[A]补=00,1110011,

[B]补=11,1101000

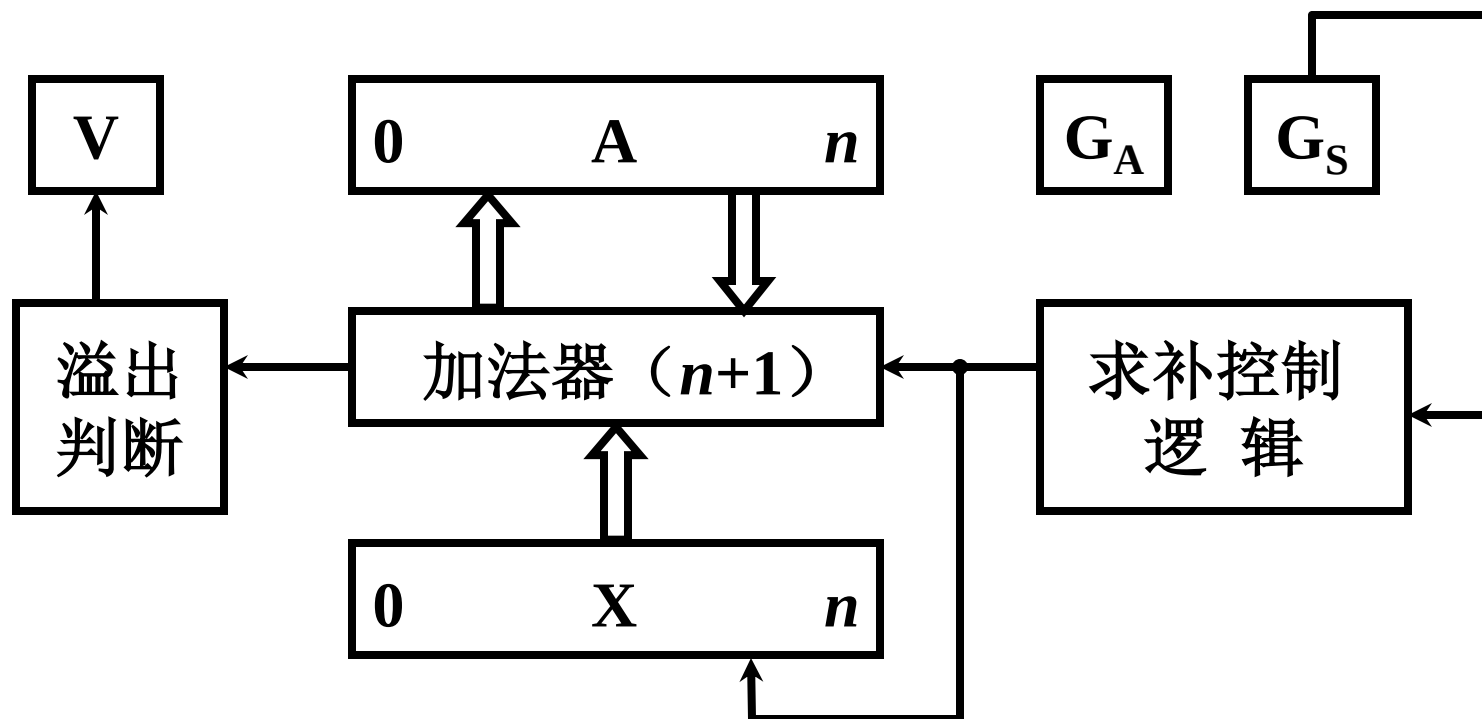
[A+B]补= 00,1110011

+ 11,1101000

= 100,1011011---无溢出

A+B= +1011011 = +91

4. 补码加减法的硬件配置



A、X 均 $n+1$ 位

用减法标记 G_S 控制求补逻辑

作业 6.4

1. 设机器数字长为 8 位（含 1 位符号位）

且 $A = 20$, $B = 26$, 用补码求 $A + B$, $A - B$

2. 设机器数字长为 6 位（含 1 位符号位）

且 $A = 20$, $B = -26$, 用补码求 $A + B$, $A - B$

要求：用两位符号位解题以及判断是否有溢出

Thank you